

AURA System Architecture and Working

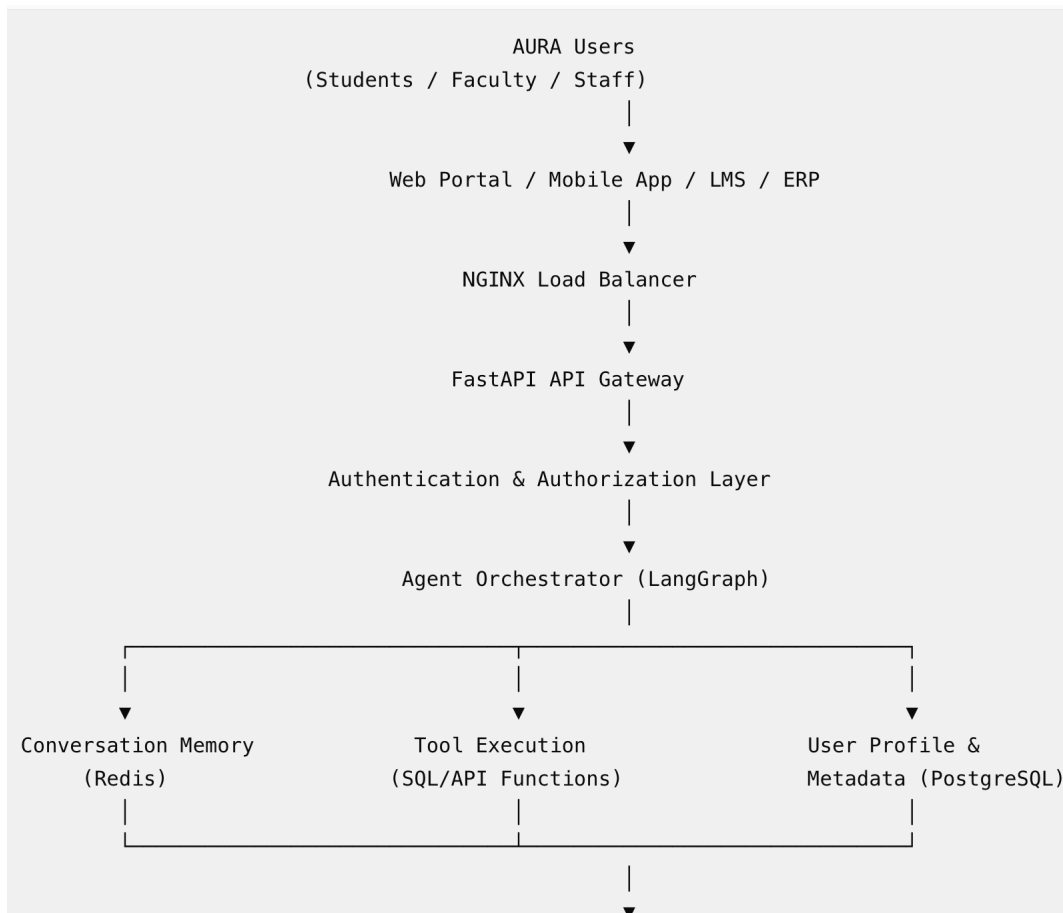
1. Introduction

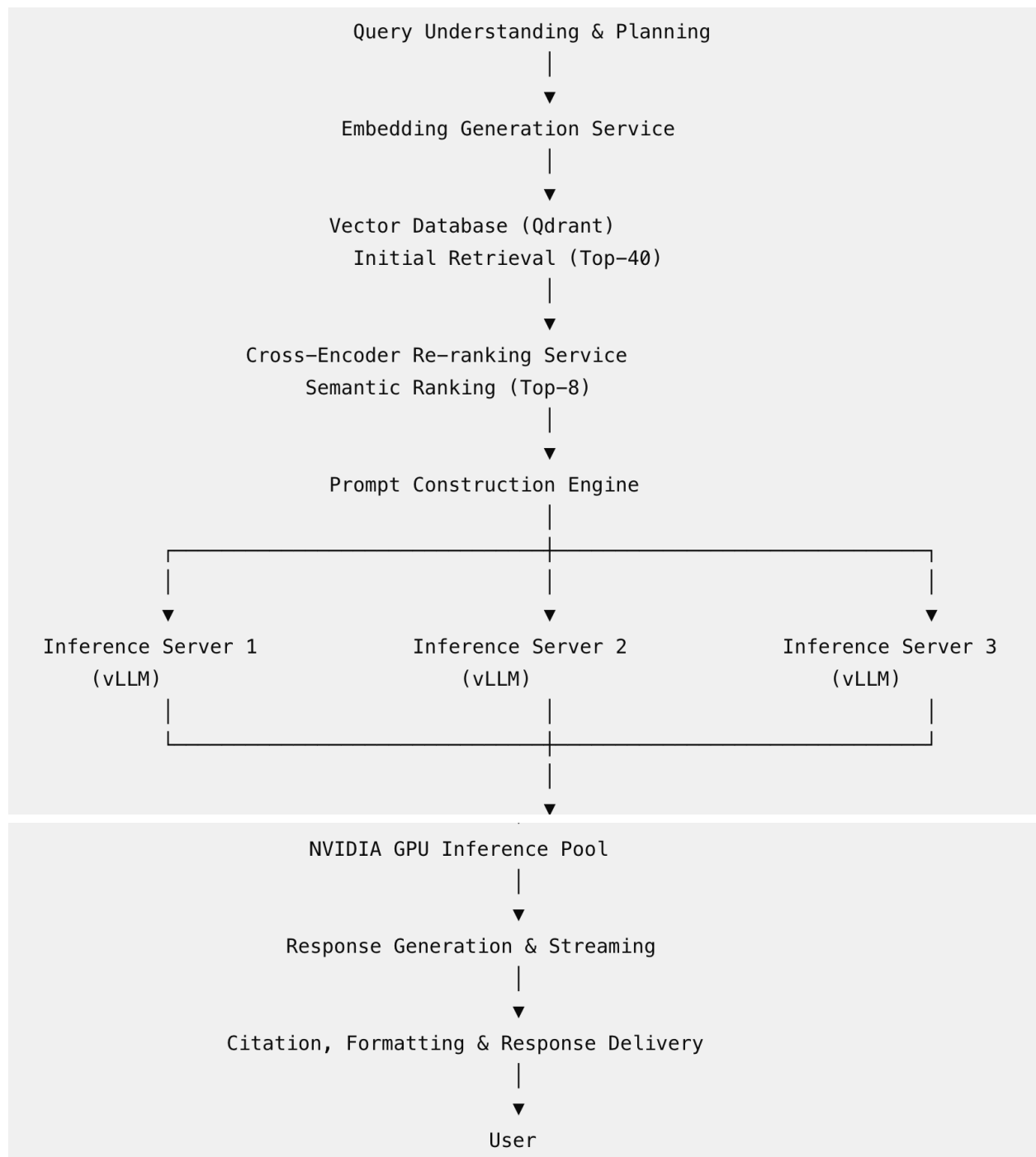
AURA (AI-powered University Resource Assistant) is an **Agentic Retrieval-Augmented Generation (Agentic RAG)** system designed to answer students' and faculty members' questions using the university's curated knowledge base.

Unlike a conventional chatbot that directly forwards every user query to a Large Language Model (LLM), AURA follows a multi-stage reasoning pipeline. Each stage performs a specialized responsibility such as authentication, semantic retrieval, reranking, reasoning, prompt construction, and language generation. This modular architecture improves accuracy, scalability, explainability, and maintainability.

The architecture follows the principle of **separation of concerns**, in which each software component performs a well-defined task. This makes the system easier to develop, debug, scale, and migrate to future infrastructure.

The complete logical architecture is shown below.





The **Inference Router** (after Prompt Construction Engine) is responsible for selecting the least-loaded inference server, retrying failed requests, and hiding the physical topology from LangGraph. That way, LangGraph only knows it is calling an "LLM service," not which Node 1, Node 2, or Node 3 is responding. This abstraction makes the system easier to scale later—adding a fourth or fifth inference node requires only updating the router configuration, not changing the orchestration logic.

Node 1 (RTX 4090)	Node 2 (RTX 4090)	Node 3 (RTX 4090)	Node 4 (RTX 5080)
Ubuntu Server 24.04	Ubuntu Server	Ubuntu Server	Ubuntu Server
Docker	Docker	Docker	Docker
NGINX	vLLM Instance #2	vLLM Instance #3	Embedding Service
FastAPI	RTX 4090	RTX 4090	Qdrant
LangGraph			Cross-Encoder
Redis			Prometheus
PostgreSQL			Grafana
vLLM Instance #1			
RTX 4090			

1.1 Query Execution Flow

Student → NGINX → FastAPI → LangGraph → Redis (previous conversation) → Embedding Service → Qdrant (Top 40) → Cross Encoder (Top 8) → Prompt Builder → Load Balancer → Available vLLM (Node1 OR Node2 OR Node3) → RTX 4090 → Generated Answer → Student

Notice that LangGraph doesn't decide which GPU to use. Instead, the inference layer should expose a single inference endpoint (or be fronted by an internal load balancer), and requests are automatically routed to whichever vLLM instance has capacity.

2. Overall Request Lifecycle

Before discussing each component individually, let us first understand what happens when a student asks a question.

Suppose a student opens the AURA web portal and types the following query:

"Can I withdraw from the M.Tech program after the first semester?"

Although the student sees only a text box, the system internally performs several coordinated operations before producing the final answer.

The request first reaches the API Gateway through the NGINX Load Balancer. After the user's identity is verified, the Agent Orchestrator analyzes the query and determines the appropriate execution plan. The user's question is then converted into a semantic vector by the Embedding Service and matched against the university's knowledge base stored in the Vector Database.

The retrieved candidate document chunks are then reranked using a Cross-Encoder model to identify the most relevant evidence. These reranked chunks, together with the user's question, conversation history, and system instructions, are assembled into a structured prompt.

The prompt is forwarded to one of the available inference servers, where the Large Language Model generates the answer using GPU acceleration. Finally, citations are attached, the response is formatted, and the answer is returned to the student.

The entire process typically completes in 2 to 4 seconds.

3. Users and Client Applications

The entry point of AURA is the user interface. Users may interact with AURA through the university web portal, mobile application, ERP system, Learning Management System (LMS), or any future client application.

For example, a student may ask:

"How many credits are required for the M.Sc. Data Science program?"

or a faculty member may ask:

"What is the travel reimbursement policy?"

Regardless of the interface used, every request follows the same backend execution pipeline. The client applications themselves contain almost no business logic. Their responsibility is limited to collecting user input, displaying streamed responses, and maintaining the user session.

4. NGINX Load Balancer

NGINX is the first backend service that receives every incoming request. Its primary responsibility is not to generate answers but to efficiently manage network traffic.

Suppose one thousand students simultaneously access AURA during the semester registration period.

If every request were routed directly to the API Gateway, the gateway could become overloaded. Instead, NGINX distributes requests across backend services while also performing HTTPS termination, request routing, and rate limiting.

For example, if one student repeatedly sends hundreds of requests every minute due to a programming error or malicious intent, NGINX can automatically reject excessive requests before they consume expensive backend resources.

NGINX also continuously monitors backend health. If one inference server becomes unavailable, requests are automatically redirected to the remaining servers without interrupting the user experience.

Thus, NGINX acts as the traffic controller of the entire platform.

5. FastAPI API Gateway

The API Gateway is the central communication hub of AURA. Every request received from NGINX is processed by FastAPI. Its responsibilities include user authentication, authorization, request validation, logging, and routing.

Suppose a faculty member asks:

"Show me the HR leave policy."

Before forwarding this request, the API Gateway verifies the user's identity through the university's Single Sign-On (SSO) system.

If a student attempts to access confidential HR documents, the request is rejected before it reaches the AI components. The API Gateway also validates uploaded parameters, records usage logs, measures response times, and forwards valid requests to the Agent Orchestrator. The gateway, therefore, acts as the secure entrance to the entire AI platform.

6. Agent Orchestrator (LangGraph)

The Agent Orchestrator is the intelligence layer of AURA. Many beginners incorrectly assume that the language model itself performs reasoning and planning. In reality, the LLM only predicts text.

The Agent Orchestrator decides **which information to collect before asking the language model**.

Consider the following question.

"Compare the DAU hostel policy with the M.Tech student handbook."

Answering this question requires multiple operations.

The orchestrator first identifies the required documents. It then invokes the semantic retrieval pipeline. If necessary, it may also query SQL databases or external APIs. Only after collecting all relevant evidence does it prepare the final prompt for the language model. The orchestrator, therefore, behaves like a project manager coordinating multiple specialized workers.

This design significantly improves answer quality by providing the language model with structured evidence rather than relying solely on its internal memory.

7. Redis Conversation Memory

AURA supports contextual conversations.

Suppose a student asks:

"Tell me about the available scholarships."

After receiving the answer, the student immediately asks:

"Which one is suitable for me?"

The second question contains no reference to scholarships. Redis stores the previous conversation so that AURA understands what "which one" refers to. Redis, therefore, acts as AURA's short-term memory.

Unlike PostgreSQL, Redis stores information entirely in memory, allowing extremely fast retrieval. Conversation history, temporary session information, authentication tokens, and workflow states are all stored inside Redis. When a conversation expires, Redis automatically removes the temporary data.

8. PostgreSQL Database

PostgreSQL stores the platform's permanent data. Unlike Redis, its contents persist even after the server is restarted. Examples include user accounts, departments, permissions, feedback, conversation analytics, audit logs, and system configuration.

Suppose the university administrator wants to know:

- How many students used AURA yesterday?
- Which departments generated the highest number of queries?
- What were the most frequently asked questions?

These analytics can be generated directly from PostgreSQL. The database, therefore, acts as the long-term memory of the entire platform.

9. Embedding Generation Service

The first AI component encountered during semantic retrieval is the Embedding Service. An embedding is a numerical representation of the meaning of a piece of text.

Suppose a student asks:

"Can I cancel my admission before classes begin?"

Although the official policy may instead use the phrase

"Students may withdraw before semester registration."

Both statements express nearly the same meaning.

The Embedding Service converts both the question and every document chunk into dense vectors. These vectors enable semantic comparison instead of exact keyword matching. Unlike the Large Language Model, the Embedding Service never generates answers. Its only responsibility is converting text into vectors.

10. Vector Database (Qdrant)

After the user's question has been converted into an embedding, the vector is sent to Qdrant. Qdrant stores embeddings corresponding to every chunk of every university document. Instead of searching for identical words, it searches for semantically similar content.

Suppose the student's query refers to "hostel refund."

Qdrant may retrieve relevant paragraphs from the Hostel Policy even if the document never contains the exact phrase "hostel refund."

Initially, Qdrant retrieves the top forty candidate chunks. These are only approximate matches. The final selection is performed by the Cross-Encoder.

11. Cross-Encoder Re-ranking Service

The Cross-Encoder is one of the most important components of AURA's retrieval pipeline. Most RAG systems stop after vector retrieval. AURA performs an additional reranking step.

Suppose Qdrant retrieves the following four candidate chunks.

- Chunk A discusses hostel fees.
- Chunk B discusses admission cancellation.
- Chunk C discusses withdrawal deadlines.
- Chunk D discusses examination rules.

Although Qdrant considers all four relevant, the Cross-Encoder reads each candidate together with the original question.

For every pair,

Question + Chunk

the Cross-Encoder predicts a relevance score.

It may be concluded that Chunk C is actually the most relevant, even though Qdrant ranked it third. Only the highest-ranked chunks are supplied to the language model. This additional reranking stage substantially improves answer accuracy.

12. Prompt Construction Engine

The Prompt Construction Engine assembles the final prompt that will be supplied to the language model.

The prompt contains several pieces of information.

These include the user's question, conversation history from Redis, reranked evidence from Qdrant, system instructions, response formatting guidelines, and citation requirements.

Instead of asking the LLM,

"Can I withdraw after Semester 1?"

The actual prompt resembles:

You are AURA, the official university assistant. Answer the following question using only the supplied institutional documents. If the answer is unavailable, clearly state so. Cite every policy reference.

This structured prompting significantly improves factual consistency.

13. Inference Router

After the prompt is constructed, the request is sent to the Inference Router. The router is responsible for selecting one of the available inference servers.

Suppose three inference servers are available. If Server 2 is currently processing many requests while Server 3 is idle, the router automatically forwards the next request to Server 3. The Agent Orchestrator does not need to know where the model is running. It simply communicates with the router, which hides the underlying infrastructure.

This design makes the system easily scalable because additional inference servers can be added without modifying the orchestration logic.

14. LLM Inference Servers

The inference servers are dedicated computers that are solely responsible for language generation. Each server hosts a copy of the selected language model using vLLM.

The inference servers perform no authentication, retrieval, reranking, or database operations. Their only responsibility is generating responses.

When the router forwards a prompt, the inference server loads it into GPU memory, runs the language model, and streams the generated tokens back.

Separating inference from orchestration ensures that GPU resources remain focused on language generation.

15. vLLM Inference Engine

Inside every inference server runs vLLM. vLLM is not another language model. It is the software that efficiently operates the language model.

Suppose twenty students simultaneously submit questions. Instead of processing them sequentially, vLLM dynamically batches multiple requests together and schedules them on the GPU.

It also streams generated tokens immediately and manages GPU memory efficiently. Without vLLM, expensive GPUs would remain underutilized.

16. NVIDIA GPU Inference Cluster

The GPUs perform the mathematical computations required by the language model. Every generated token requires billions of matrix multiplications and attention operations. The GPUs neither understand policies nor perform reasoning. Their sole responsibility is executing the numerical computations required by the language model.

During AURA's temporary deployment, three NVIDIA RTX 4090 GPUs are used for inference. Together, they form the GPU inference pool serving multiple users simultaneously.

17. Response Generation and Citation Layer

Once the language model finishes generating the answer, the response is returned to the Agent Orchestrator.

Before sending it to the user, AURA performs several post-processing operations. The response is formatted into Markdown or HTML. Citations corresponding to the retrieved document chunks are attached. Tables, lists, or hyperlinks are generated if required. The updated conversation history is stored in Redis. Finally, response metadata and usage logs are recorded in PostgreSQL. Only then is the completed response returned to the user.

[Complete Working Example](#)

To better understand the interaction among all components, consider the following query.

Student: "Can I withdraw from the M.Tech program after the first semester?"

The request first reaches NGINX, which forwards it to the FastAPI API Gateway after validating the connection. The API Gateway authenticates the student and forwards the request to the Agent Orchestrator.

The orchestrator retrieves the student's conversation context from Redis and determines that this query requires semantic retrieval from the institutional policy repository. The question is converted into an embedding by the Embedding Service and searched against Qdrant, which retrieves the top forty candidate chunks. The Cross-Encoder reranks these candidates and selects the eight most relevant passages.

The Prompt Construction Engine combines the user's question, the retrieved evidence, the conversation history, and system instructions into a single structured prompt. The Inference Router forwards this prompt to the least-loaded inference server. Inside that server, vLLM schedules the request on one of the RTX 4090 GPUs, where the language model generates the answer token by token.

The generated response is returned to the Agent Orchestrator, which attaches policy citations, formats the response, updates Redis with the latest conversation context, records analytics in PostgreSQL, and sends the final answer back to the student.

From the student's perspective, the interaction appears to be a simple question-and-answer exchange. Internally, however, AURA coordinates more than a dozen specialized software components working together to produce an accurate, context-aware, and citation-backed response. This modular design is what enables AURA to scale reliably to thousands of users while remaining maintainable and portable to future infrastructure.

Containerization and Deployment Strategy

AURA is designed as a **containerized microservice-based application**. Instead of installing every software component directly on the operating system, each component executes inside its own Docker container. This design ensures that every service remains isolated from the others while communicating through well-defined network interfaces.

The primary motivation behind adopting containerization is portability. The current AURA deployment is intended to run on a temporary four-node GPU cluster. In the future, when a dedicated AI server or cloud infrastructure becomes available, the entire application should be transferable without requiring any modifications to the application code. By containerizing every service, migration becomes a deployment task rather than a software development task.

Instead of treating the physical machines as the primary unit of deployment, AURA treats each software component as an independent service. The operating system simply provides the execution environment, while Docker provides isolated runtime environments for every service.

For example, the FastAPI API Gateway, Redis, PostgreSQL, Qdrant, LangGraph, Embedding Service, Cross-Encoder Service, and every vLLM inference engine execute as separate Docker containers.

Conceptually, the deployment resembles the following:

```
Machine
|
├─ Docker Engine
|
├─ nginx container
├─ fastapi container
├─ langgraph container
├─ redis container
├─ postgres container
├─ qdrant container
├─ embedding container
├─ reranker container
├─ vllm container
└─ monitoring container
```

Each container contains everything required to execute that particular service, including its operating environment, libraries, dependencies, configuration files, and runtime.

Consequently, installing Python packages or AI libraries directly on the host operating system is unnecessary. Every dependency remains encapsulated inside its respective container.

This isolation eliminates dependency conflicts between services. For instance, if the Embedding Service requires one version of PyTorch while the Cross-Encoder requires another, both services can coexist without interfering with one another because they run in separate containers.

Why Docker Compose?

Since the temporary deployment consists of only four servers, introducing Kubernetes would unnecessarily increase operational complexity.

Instead, AURA uses **Docker Compose** for orchestration.

Docker Compose allows multiple related containers to be described in a single declarative configuration file (`docker-compose.yml`). Rather than manually starting each service one by one, the entire platform can be launched using a single command:

</> Bash

```
docker compose up -d
```

Similarly, shutting down the entire platform requires only:

</> Bash

```
docker compose down
```

The Compose file specifies:

- which containers should be created,
- which Docker images should be used,
- which ports should be exposed,
- how services communicate,
- which persistent storage volumes are mounted,
- restart policies,
- environment variables,
- network configuration,
- service dependencies.

Thus, the complete AURA platform becomes **Infrastructure as Code (IaC)**. The deployment itself is version-controlled and reproducible.

Containerization of AURA Services

Each major software component of AURA executes inside an independent Docker container. A representative deployment is shown below.

Service	Docker Container
NGINX	<code>aura-nginx</code>

FastAPI Gateway	<code>aura-api</code>
LangGraph	<code>aura-orchestrator</code>
Redis	<code>aura-redis</code>
PostgreSQL	<code>aura-postgres</code>
Qdrant	<code>aura-qdrant</code>
Embedding Service	<code>aura-embedding</code>
Cross-Encoder Service	<code>aura-reranker</code>
vLLM Server 1	<code>aura-vllm-node1</code>
vLLM Server 2	<code>aura-vllm-node2</code>
vLLM Server 3	<code>aura-vllm-node3</code>
Prometheus	<code>aura-prometheus</code>
Grafana	<code>aura-grafana</code>

Each service can be independently started, stopped, upgraded, restarted, or replaced without affecting the remaining components of the platform.

Docker Networking

Although the services run in different containers—and in some cases on different physical machines—they communicate as if they were part of a single application.

For example, the Agent Orchestrator does not communicate with a physical server named "Node 4." Instead, it communicates with a service endpoint such as:

<http://embedding-service:8000> OR <http://reranker-service:8001>

Similarly, FastAPI communicates with Redis using the service name:

`redis:6379`

rather than relying on hard-coded IP addresses whenever possible.

This abstraction decouples the software architecture from the underlying hardware architecture.

Persistent Storage

Docker containers are intentionally ephemeral.

If a container is deleted, all data stored in it is lost. Therefore, AURA stores persistent information using Docker Volumes.

For example,

- PostgreSQL data
- Redis persistence (if enabled)
- Qdrant vector indexes
- System logs
- Configuration files

are stored outside the containers.

A typical storage layout is:

Host Storage

```
|  
├─ postgres-data/  
├─ redis-data/  
├─ qdrant-storage/  
├─ logs/  
└─ backups/
```

When a container is recreated, these directories are automatically mounted, allowing the service to resume operation without losing data.

Environment Configuration

One of AURA's guiding principles is that **no configuration should be hard-coded in the application**.

Instead, all configurable parameters are supplied through environment variables.

Typical configuration parameters include:

- database credentials,
- API keys,
- model names,
- service URLs,

- GPU assignments,
- network ports,
- logging levels.

For example, a common `.env` file may contain:

```
POSTGRES_HOST=postgres
```

```
POSTGRES_PORT=5432
```

```
REDIS_HOST=redis
```

```
QDRANT_HOST=qdrant
```

```
EMBEDDING_SERVICE=http://embedding-service:8000
```

```
RERANKER_SERVICE=http://reranker-service:8001
```

```
LLM_ENDPOINT=http://inference-router:9000
```

Changing the deployment from the temporary four-node cluster to a future production server requires updating only these configuration values rather than modifying application code.

Portability of the Entire Platform

One of the major advantages of containerization is that the complete AURA platform becomes hardware-independent.

Suppose the current deployment executes on four separate GPU workstations. Nodes 1...4.

When the university later provides a dedicated AI server containing multiple GPUs, no application redesign is necessary.

The deployment process simply becomes:

1. Install Ubuntu Server.
2. Install Docker.
3. Install Docker Compose.
4. Copy the AURA project repository.
5. Copy the `.env` configuration file.
6. Restore PostgreSQL and Qdrant backups.
7. Execute:

</> Bash

```
docker compose up -d
```

Within a few minutes, the complete AURA platform becomes operational on the new hardware.

The application code, orchestration logic, retrieval pipeline, reranking pipeline, and API interfaces remain completely unchanged.

Future Migration to Kubernetes

The current deployment intentionally uses Docker Compose because it provides an excellent balance between simplicity and functionality for a four-node infrastructure.

However, because every service already executes as an independent container, the architecture naturally supports future migration to Kubernetes.

When AURA grows to support larger user populations or additional GPU servers, the migration process primarily involves replacing Docker Compose with Kubernetes manifests (or Helm charts). Since the application is already decomposed into independent microservices, Kubernetes can take over responsibilities such as service discovery, automatic scaling, rolling updates, self-healing, and high availability without requiring significant changes to the application itself.

Thus, the current Docker Compose–based deployment should be viewed as the first step toward a production-grade cloud-native architecture rather than a temporary workaround.

Architectural Philosophy

The deployment strategy of AURA follows a simple but powerful principle:

Build services, not servers.

Servers are temporary and may change over time as hardware evolves or infrastructure is upgraded. Services, however, represent the functional building blocks of the system. By encapsulating all components in Docker containers and orchestrating them with Docker Compose, AURA becomes portable, reproducible, maintainable, and future-ready.

This architectural philosophy ensures that migrating from the current four-node experimental cluster to a dedicated institutional AI server—or even to a cloud or Kubernetes environment—requires minimal operational effort while preserving the same software architecture and application behavior.

Capacity Planning, Parallel Request Handling, and Response Time Estimation

Designing an AI system involves not only ensuring that it produces correct answers but also that it can respond to many users simultaneously. Unlike traditional web applications, the computational cost of a Large Language Model (LLM) is several orders of magnitude higher than that of a typical database query or REST API call. Consequently, the system's overall throughput is primarily determined by the inference servers and GPUs rather than by the API Gateway or databases.

Before deploying AURA, it is essential to estimate the number of parallel requests that can be served and the expected user response time.

The calculations presented in this section are based on the temporary deployment infrastructure used for AURA and should be regarded as engineering estimates rather than absolute limits. The same methodology can be applied to future hardware configurations by replacing the relevant parameters.

Hardware Configuration

The temporary deployment consists of four GPU workstations connected through a high-speed local network.

Node	Hardware Role	GPU
Node 1	Gateway + Orchestrator + Inference	RTX 4090
Node 2	Inference	RTX 4090
Node 3	Inference	RTX 4090
Node 4	Retrieval Services	RTX 5080

Each workstation contains:

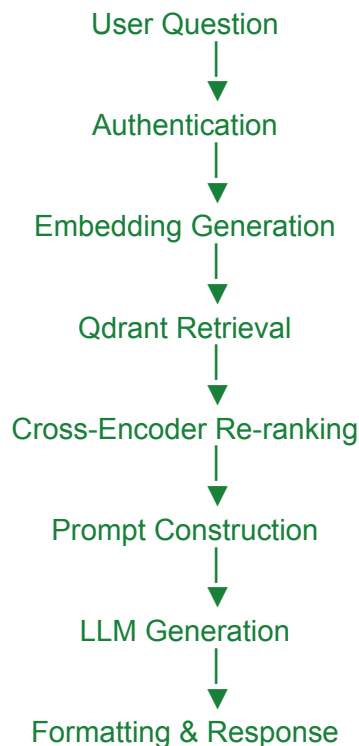
- 64 GB DDR5 RAM
- 2 TB NVMe SSD
- Ubuntu Server 24.04
- Docker Engine
- Docker Compose

Three GPUs participate in language generation, while the RTX 5080 executes retrieval-related AI services such as embedding generation and Cross-Encoder reranking.

Understanding the Critical Path

When a user submits a question, not every software component contributes equally to the overall response time.

The complete execution pipeline is



Among these stages, **LLM generation is by far the most computationally expensive operation.**

Operations such as authentication, Redis access, PostgreSQL queries, Qdrant retrieval, and prompt construction typically complete within a few milliseconds. In contrast, generating several hundred output tokens with a 30+ billion-parameter language model requires billions of floating-point operations and consumes most of the total response time.

Consequently, AURA's throughput is primarily determined by the inference layer.

Estimating the Cost of a Single Request

Consider the following representative query.

"Can I withdraw from the M.Tech program after the first semester?"

Assume the following prompt characteristics.

Parameter	Value
User question	25 tokens
Conversation history	500 tokens
Retrieved context	1,500 tokens
System prompt	500 tokens
Total input context	≈ 2,500 tokens
Expected answer	≈ 500 tokens

Thus, the language model processes approximately **2,500 input tokens + 500 generated output tokens** for a total context of approximately **3,000 tokens**.

Throughput of a Single RTX 4090

For planning purposes, assume that the selected quantized language model (e.g., Qwen3-32B) running under vLLM generates approximately **300 output tokens per second** on an RTX 4090.

This value varies depending on the model, quantization method, context length, batching strategy, and decoding parameters. Nevertheless, it provides a realistic engineering estimate.

- Generating 500 output tokens, therefore, requires approximately $500 / 300 \approx 1.7$ seconds.
- Prompt processing introduces additional latency. Including prompt encoding and scheduling overhead, the total inference time becomes approximately ≈ 2.5 seconds for a typical query.

Why Parallel Requests Are Possible

A common misconception is that a GPU can process only one request at a time.

This is not how modern inference engines operate.

AURA uses a **vLLM** that employs continuous batching. Instead of executing requests sequentially, vLLM dynamically groups multiple requests into a single GPU execution batch.

Suppose four students submit questions almost simultaneously.

A naïve inference server would execute them one after another, for example, Request 1 → Request 2 → Request 3 → Request 4. This approach leaves the GPU underutilized because the hardware spends considerable time waiting between requests.

In contrast, vLLM combines these requests into a shared execution batch. {Request 1, Request 2, Request 3, Request 4} → Dynamic Batch → GPU Execution. By continuously admitting new requests into the batch as previous ones complete, vLLM maintains high GPU utilization while reducing the average response time.

Parallel Capacity of One Inference Server

For interactive university applications, users generally expect responses within approximately three to five seconds.

Based on practical experience with RTX 4090 GPUs running quantized 30–32 billion parameter models under vLLM, a single inference server can comfortably sustain approximately **20–30 simultaneously active inference requests** without excessive latency.

For capacity planning, AURA adopts the conservative value of **25 concurrent requests per GPU**. This value provides a good balance between throughput and user experience.

Capacity of the Complete Inference Layer

The temporary deployment contains three independent inference servers. Therefore,

$$\text{Concurrent Requests} = \text{Inference Servers} \times \text{Requests per Server} = 3 \times 25 = 75$$

Hence, approximately **75 language generation requests** can be actively processed in parallel before requests begin accumulating in the inference queue.

End-to-End Response Time

Although LLM generation dominates the execution time, the retrieval pipeline also contributes to the total latency.

Typical execution times are shown below.

Processing Stage	Typical Time
Authentication	5–10 ms
Redis Memory Access	2–5 ms
Embedding Generation	20–40 ms
Qdrant Retrieval	10–20 ms
Cross-Encoder Re-ranking	80–150 ms
Prompt Construction	10–20 ms
Queue Waiting (vLLM)	0–500 ms
LLM Generation	2.0–3.0 s
Response Formatting	20–50 ms

Therefore, the typical end-to-end response time experienced by users is approximately **2.3 to 3.8 seconds**. Even under moderate load, responses remain comfortably within the interactive range expected for conversational assistants.

Estimating Requests per Second

If one inference request occupies a GPU for approximately ≈ 3 seconds and each GPU can maintain ≈ 25 active requests,

Then one inference server completes approximately $25 / 3 \approx 8.3$ requests per second

With three inference servers, $3 \times 8.3 \approx 25$ requests per second

Consequently, the temporary deployment can sustain approximately **25 complete AI requests every second** under continuous load.

Estimating Conversation Throughput

Users generally ask more than one question during a conversation. Assume that a typical interaction contains **four conversational turns**.

Then, 25 requests per second $\rightarrow 1500$ requests per minute $\rightarrow 1500 / 4$
 ≈ 375 complete conversations per minute

Thus, AURA can theoretically support approximately **375 complete multi-turn conversations every minute** under continuous peak load.

Can AURA Serve 3,000 Users?

It is important to distinguish between **registered users**, **active users**, and **concurrent requests**.

Suppose the university has

- 3,000 registered users,
- approximately 300 users online simultaneously,
- and only 25% of those users are actively waiting for AI responses at any instant.

Then, $300 \times 0.25 = 75$ concurrent requests.

This value closely matches the estimated capacity of the temporary deployment.

In practice, university usage patterns are highly bursty. Peaks occur during admission, examination, registration, and result declaration periods, while normal academic hours exhibit significantly lower request rates. Under typical conditions, the proposed architecture should comfortably support university-wide deployment.

What Happens When More Than 75 Requests Arrive?

The estimate of 75 concurrent requests does **not** represent a hard system limit or the maximum number of users AURA can support. Rather, it indicates the approximate number of inference requests that can be executed concurrently while maintaining the target response latency.

If additional requests arrive while all three inference servers are fully occupied, they are **not rejected**. Instead, they are placed into the scheduling queue maintained by **vLLM** (and, at a higher level, by the inference routing layer). As soon as one of the active requests finishes generating its response, the next queued request is admitted for execution.

For example, suppose 100 users submit questions simultaneously.

- The first **75 requests** are immediately dispatched to the three inference servers.
- The remaining **25 requests** are added to the waiting queue.
- As inference completes, queued requests are continuously scheduled onto the available GPUs.

This queuing mechanism ensures that every valid request is eventually processed, although users joining the queue during periods of heavy demand may observe slightly longer response times.

Scalability of the Architecture

One of the major advantages of AURA's microservice-based design is that the inference layer can be expanded independently of the rest of the system components.

Suppose two additional GPU servers are added in the future.

The new capacity becomes

$$5 \text{ inference servers} \times 25 \text{ concurrent requests} = 125 \text{ concurrent requests}$$

Notice that **no changes are required to the FastAPI Gateway, LangGraph orchestration, Redis, PostgreSQL, or Qdrant**. The Inference Router simply begins forwarding requests to the two additional inference servers.

This linear scaling capability is one of the primary motivations for separating orchestration, retrieval, and inference into independent services. It allows the university to increase AURA's serving capacity incrementally as demand grows without redesigning the overall architecture.

Engineering Note

The numerical estimates presented in this section are intentionally conservative. Actual throughput depends on several factors, including the chosen language model, quantization method, context length, decoding strategy, GPU driver version, and batching efficiency. The calculations should therefore be interpreted as **capacity-planning guidelines**, providing a sound basis for infrastructure sizing and performance expectations rather than as fixed performance guarantees. This methodology can be reused whenever the hardware, models, or deployment configuration changes.